



Las Americas Institute of Technology

Nombre del curso

Programación Paralela

Título del proyecto

Analizador de Almacenamiento Paralelo

Integrantes del equipo

Rubby Keyther Martinez

Eidan Yamil Then Elizo

Nombre del líder

Jeremy Reyes Gonzalez

Fecha de entrega

22/4/2026

Índice

Analizador de Almacenamiento Paralelo.....	3
1. Introducción	3
2. Descripción del Problema	4
3. Cumplimiento de los Requisitos del Proyecto	5
4. Diseño de la Solución.....	6
5. Implementación Técnica	12
6. Evaluación de Desempeño	13
7. Trabajo en Equipo	15
8. Conclusiones	16
9. Referencias	17
10. Anexos.....	17

Analizador de Almacenamiento Paralelo

1. Introducción

Presentación general del proyecto.

El proyecto consiste en el desarrollo de una herramienta de consola diseñada para la auditoría y limpieza de sistemas de archivos. La aplicación combina dos funcionalidades críticas: la identificación de archivos que superan un tamaño definido por el usuario y la detección de archivos duplicados mediante firmas digitales (**hashes**).

Para procesar volúmenes masivos de datos, el sistema emplea **descomposición recursiva** y procesamiento paralelo, permitiendo reducir drásticamente el tiempo de escaneo en comparación con las herramientas tradicionales del sistema operativo.

Justificación del tema elegido.

El crecimiento exponencial de la información digital ha generado un aumento significativo en el uso de almacenamiento en sistemas informáticos. En este contexto, es común que los usuarios acumulen archivos duplicados y archivos de gran tamaño que ocupan espacio innecesariamente, afectando el rendimiento del sistema y dificultando la gestión eficiente de los recursos.

La identificación manual de estos archivos resulta ineficiente, especialmente cuando se trabaja con grandes volúmenes de datos. Por esta razón, surge la necesidad de desarrollar una herramienta automatizada que permita analizar el sistema de archivos de manera rápida y precisa.

Además, el proyecto incorpora técnicas de procesamiento paralelo y descomposición recursiva, lo cual permite aprovechar al máximo los recursos del hardware moderno, reduciendo considerablemente los tiempos de ejecución en comparación con métodos secuenciales tradicionales. Esto no solo resuelve un problema práctico del mundo real, sino que también permite aplicar conceptos avanzados de programación concurrente en un entorno realista.

Objetivo General.

Desarrollar una aplicación de consola capaz de analizar el sistema de archivos de un equipo, identificando archivos de gran tamaño y detectando archivos duplicados mediante el uso de técnicas de procesamiento secuencial y paralelo, con el fin de optimizar el uso del almacenamiento.

Objetivos Específico.

Implementar un sistema de exploración de directorios utilizando descomposición recursiva para recorrer estructuras de carpetas de manera eficiente.

Desarrollar un módulo que identifique archivos que superen un tamaño mínimo definido por el usuario.

Diseñar e implementar un algoritmo para la detección de archivos duplicados mediante el uso de funciones hash.

Aplicar técnicas de programación paralela para mejorar el rendimiento del procesamiento de grandes volúmenes de datos.

Comparar el desempeño entre la ejecución secuencial y paralela mediante métricas como tiempo de ejecución, speedup y eficiencia.

Desarrollar una interfaz de consola que permita la interacción con el usuario de forma clara y funcional.

2. Descripción del Problema

Contexto del problema seleccionado

La gestión manual del almacenamiento es ineficiente en sistemas modernos. Los problemas principales son:

- **Desperdicio de recursos:** Archivos grandes olvidados (instaladores, ISOs, logs) y copias redundantes de los mismos datos.
- **Ineficiencia Secuencial:** Recorrer un árbol de directorios con millones de archivos de forma lineal es extremadamente lento, ya que no aprovecha los múltiples núcleos de los CPUs actuales.
- **Dificultad de Identificación:** Dos archivos con nombres distintos pueden ser idénticos en contenido; solo un análisis de integridad (Hash) puede confirmarlo, pero este proceso es costoso en términos de cómputo.

Aplicación del problema en un escenario real.

Este problema es común en computadoras personales y servidores, donde se acumulan descargas, copias de archivos, respaldos y registros del sistema. La falta de herramientas eficientes para identificar estos archivos provoca desperdicio de espacio y dificulta la gestión del sistema.

Importancia del paralelismo en la solución.

El uso de procesamiento paralelo permite analizar múltiples archivos y carpetas simultáneamente, reduciendo significativamente el tiempo de ejecución. Esto es fundamental cuando se trabaja con grandes volúmenes de datos, ya que aprovecha los múltiples núcleos del procesador para mejorar el rendimiento del sistema.

3. Cumplimiento de los Requisitos del Proyecto

Ejecución simultánea de múltiples tareas

El sistema utiliza la **Task Parallel Library (TPL)** para procesar múltiples carpetas y archivos al mismo tiempo. Mientras un hilo del CPU recorre una rama del directorio buscando archivos grandes, otros hilos calculan simultáneamente los hashes MD5/SHA de archivos en otras ramas para verificar duplicados.

Necesidad de compartir datos entre tareas

- Para almacenar la lista de archivos que superan el tamaño mínimo.
- Para agrupar rutas de archivos según su firma Hash. La sincronización es vital aquí para evitar condiciones de carrera al insertar nuevos duplicados desde diferentes hilos.

Estrategia de paralelización: Descomposición Recursiva

El sistema implementa una **descomposición recursiva** del árbol de directorios:

- Cada carpeta se considera un problema que se divide en sub-problemas (sus subcarpetas).
- Se lanzan tareas paralelas para explorar cada subcarpeta de forma independiente.
- La recursión se detiene al alcanzar carpetas sin subniveles, unificando los resultados de forma ascendente.

Capacidad de escalar con más recursos

La herramienta utiliza `Environment.ProcessorCount` para ajustar el grado de paralelismo. En un servidor con 32 núcleos, la capacidad de procesar archivos y calcular hashes en paralelo escalará proporcionalmente, permitiendo analizar discos empresariales en una fracción del tiempo.

Incluir métricas de evaluación del rendimiento

- **Tiempo total de escaneo:** Comparativa entre modo secuencial y paralelo.
- **Speedup y Eficiencia:** Cálculo del factor de aceleración obtenido según el número de hilos utilizados.

Aplicación a escenarios del mundo real

- **Mantenimiento de Servidores:** Identificación de backups antiguos y logs duplicados.
- **Optimización de Estaciones de Trabajo:** Limpieza de carpetas de descargas y proyectos duplicados.
- **Auditoría de Datos:** Verificación de integridad y redundancia en sistemas de archivos compartidos.

4. Diseño de la Solución

Arquitectura general del sistema

Carpeta src.

Clase ConsoleUI (Capa de Interfaz de Usuario)

La clase `ConsoleUI` se encarga de toda la interacción con el usuario a través de la consola. Gestiona la entrada de datos como la ruta a analizar, el tamaño mínimo de archivos, el número de núcleos y el modo de ejecución (secuencial, paralelo o comparativo).

Además, es responsable de mostrar los resultados mediante distintos dashboards, incluyendo la lista de archivos encontrados, los grupos de duplicados y las métricas de rendimiento (tiempo de ejecución, speedup y eficiencia). También incorpora funcionalidades para la eliminación de archivos individuales y duplicados, así como una animación tipo *spinner* para indicar procesos en ejecución.

Utiliza estructuras como `List<FileInfo>` y métodos auxiliares para formatear datos (por ejemplo, conversión de bytes a MB/GB), mejorando la legibilidad de la información mostrada.

Clase BuscadorArchivo (Capa de Procesamiento de Archivos)

La clase `BuscadorArchivo` se encarga de recorrer el sistema de archivos y recolectar todos los archivos que cumplen con un tamaño mínimo especificado. Implementa dos modos de ejecución: secuencial y paralelo.

En el modo secuencial, utiliza recursividad tradicional para recorrer directorios y subdirectorios. En el modo paralelo, emplea `Parallel.ForEach` junto con `ParallelOptions` para distribuir el procesamiento entre múltiples núcleos del procesador, mejorando el rendimiento en sistemas con múltiples hilos.

Para almacenar los resultados utiliza `ConcurrentBag<FileInfo>`, una estructura segura para entornos concurrentes. También integra `Stopwatch` para medir el tiempo de ejecución, permitiendo posteriormente calcular métricas de rendimiento.

Clase DetectorDuplicados (Capa de Análisis de Contenido)

La clase `DetectorDuplicados` tiene como objetivo identificar archivos duplicados dentro del conjunto previamente recolectado. Para ello, aplica una estrategia en dos niveles: primero agrupa los archivos por tamaño y luego por contenido mediante el uso de hashes.

El sistema utiliza el algoritmo de cifrado SHA-256 (`SHA256.Create()`) para generar una huella única de cada archivo, lo que permite comparar su contenido de forma precisa.

Al igual que la búsqueda de archivos, esta clase soporta ejecución secuencial y paralela. En el modo paralelo, emplea `Parallel.ForEach` y `ConcurrentDictionary` para gestionar los datos de forma segura en entornos concurrentes. También se controla el grado de paralelismo mediante la cantidad de núcleos configurados por el usuario.

Finalmente, los resultados se almacenan como listas de archivos duplicados (`List<List<FileInfo>>`), facilitando su posterior visualización o eliminación.

Clase ResultadoBusqueda (Capa de Transferencia de Datos)

La clase `ResultadoBusqueda` actúa como una estructura central para almacenar y transportar los resultados generados durante la ejecución del sistema. Su función principal es servir como contenedor de información entre las distintas etapas del proceso (búsqueda de archivos, detección de duplicados y visualización).

Incluye una lista de archivos encontrados (`List<FileInfo>`) y una lista de grupos de duplicados (`List<List<FileInfo>>`), lo que permite organizar claramente los resultados para su posterior análisis o manipulación. Además, almacena métricas clave como el tiempo total de ejecución (`TiempoMs`) y la cantidad de núcleos utilizados (`Nucleos`), fundamentales para evaluar el rendimiento del sistema.

También contiene información sobre el modo de ejecución (`Modo`), diferenciando entre secuencial y paralelo, junto con una propiedad derivada (`ModoNombre`) que facilita la presentación de estos datos de forma más comprensible en la interfaz.

En conjunto, esta clase permite desacoplar la lógica de procesamiento de la presentación de resultados, favoreciendo una arquitectura más organizada y mantenible.

Clase Metrics (Capa de Evaluación de Rendimiento)

La clase `Metrics` se encarga de ejecutar y comparar el rendimiento del sistema en sus dos modos de operación: secuencial y paralelo. Su función principal es centralizar la medición de desempeño, permitiendo analizar el impacto real del paralelismo implementado.

Para ello, utiliza instancias de las clases `BuscadorArchivo` y `DetectorDuplicados`, ejecutando el flujo completo del sistema en ambos modos bajo las mismas condiciones (misma ruta, tamaño mínimo y número de núcleos). Esto garantiza una comparación justa y consistente.

El método `EjecutarComparacion` coordina todo el proceso, almacenando los resultados en dos objetos `ResultadoBusqueda` (uno para cada modo), los cuales contienen tanto los datos procesados como las métricas de tiempo. Además, emplea `Task` y ejecución asíncrona para no bloquear la interfaz, junto con el uso de un *spinner* visual que mejora la experiencia del usuario durante la ejecución.

Finalmente, esta clase permite obtener la información necesaria para calcular métricas como **speedup** y **eficiencia**, facilitando la evaluación del rendimiento del sistema paralelo frente al secuencial.

Archivo Program (Capa de Control y Orquestación)

El archivo `Program` representa el punto de entrada del sistema y se encarga de coordinar el flujo completo de ejecución. Su función principal es integrar todas las clases del sistema (`ConsoleUI`, `BuscadorArchivo`, `DetectorDuplicados` y `Metrics`) y dirigir la secuencia de operaciones desde la entrada de datos hasta la visualización de resultados.

Gestiona un bucle principal que permite al usuario realizar múltiples análisis sin reiniciar la aplicación. En cada iteración, solicita los parámetros necesarios (ruta, tamaño mínimo, modo de ejecución y núcleos), delegando la interacción con el usuario a la clase `ConsoleUI`.

Dependiendo del modo seleccionado, el programa ejecuta diferentes escenarios: en modo normal (secuencial o paralelo), realiza la búsqueda de archivos y posteriormente la detección de duplicados; en modo comparativo, utiliza la clase `Metrics` para ejecutar ambos enfoques y mostrar métricas como tiempo de ejecución, **speedup** y **eficiencia**.

El sistema hace uso de programación asíncrona (`Task`, `async/await`) junto con animaciones tipo *spinner* para mejorar la experiencia del usuario durante procesos largos, evitando bloqueos en la interfaz.

Finalmente, el programa ofrece un menú interactivo que permite eliminar archivos individuales o duplicados, repetir el análisis o finalizar la ejecución, manteniendo un control completo del ciclo de vida de la aplicación.

Carpeta Test.

Clase FlujoPrincipalTests (Capa de Pruebas del Sistema)

La clase `FlujoPrincipalTests` se encarga de validar el funcionamiento integral del sistema mediante pruebas automatizadas. Su propósito principal es simular el comportamiento real del usuario y verificar que cada parte del flujo principal responda correctamente.

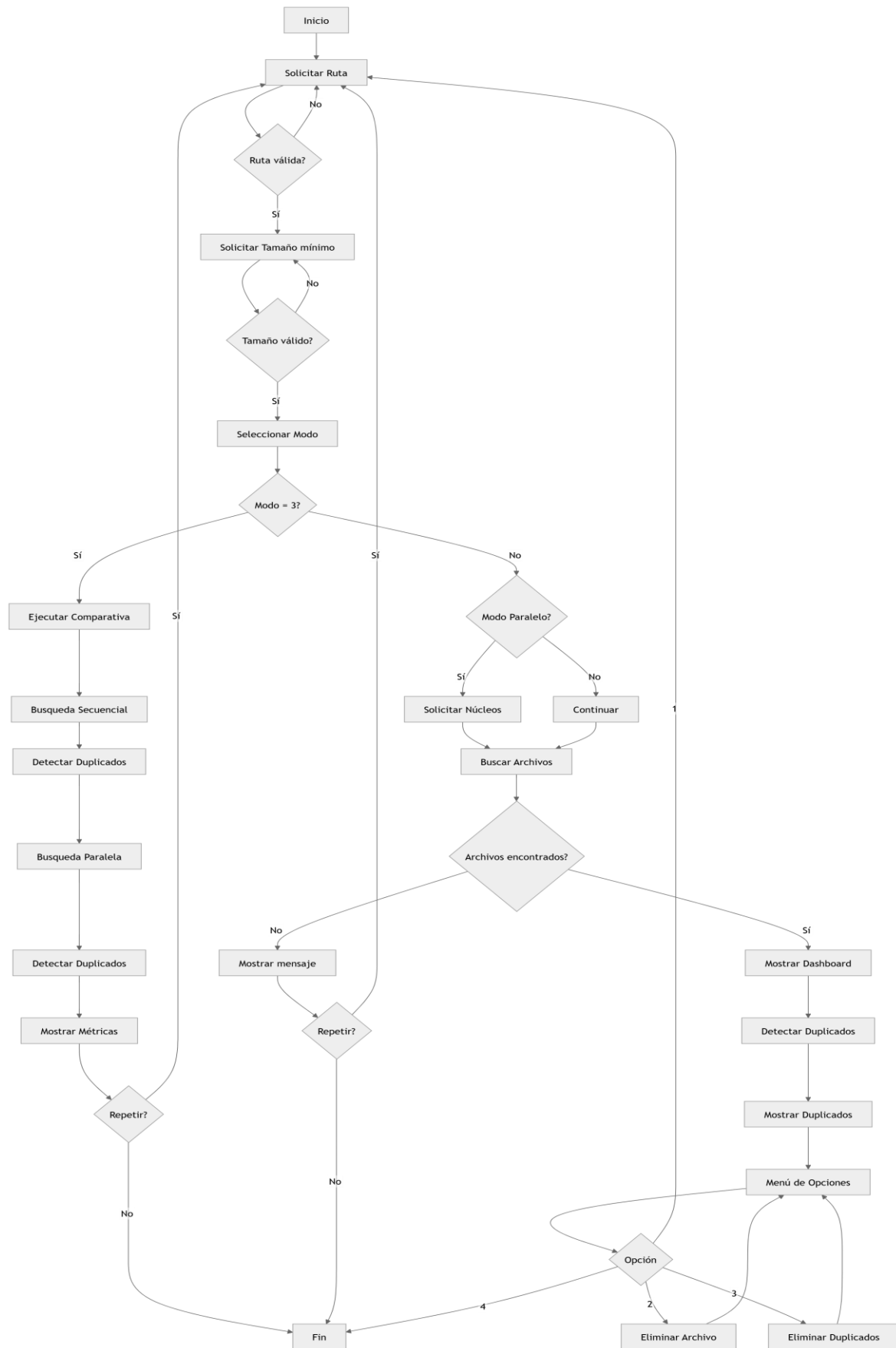
Esta clase utiliza un enfoque de pruebas basado en consola, manipulando la entrada y salida estándar (`Console.SetIn` y `Console.SetOut`) para simular interacciones del usuario sin necesidad de intervención manual. De esta manera, se pueden automatizar escenarios completos como si el usuario estuviera utilizando el sistema.

Incluye pruebas unitarias que validan tanto entradas individuales (como la captura de ruta, tamaño o modo de ejecución) como flujos completos del programa, incluyendo búsqueda de archivos, detección de duplicados y procesos de eliminación. Esto permite asegurar que el sistema funcione correctamente en situaciones reales.

Además, se emplean rutas y parámetros de prueba controlados, lo que garantiza consistencia en los resultados. También se valida el comportamiento ante decisiones del usuario, como confirmar o cancelar eliminaciones, verificando que el sistema actúe de forma segura y coherente.

Finalmente, la implementación de `IDisposable` permite restaurar el estado original de la consola después de cada prueba, evitando interferencias entre ejecuciones. En conjunto, esta clase garantiza la calidad, estabilidad y confiabilidad del sistema.

Diagrama de componentes/tareas paralel



Estrategia de paralelización utilizada

Descomposición Recursiva

El sistema implementa una estrategia de **descomposición recursiva del árbol de directorios**, donde el problema principal (analizar una ruta completa) se divide en múltiples subproblemas más pequeños de forma natural. Cada carpeta es tratada como una unidad independiente de trabajo, la cual puede descomponerse nuevamente en sus subcarpetas.

A partir de este enfoque, cada directorio se procesa y, al mismo tiempo, genera nuevas tareas para sus subdirectorios. Estas tareas se ejecutan de forma concurrente, permitiendo que diferentes ramas del árbol de archivos sean exploradas en paralelo. De esta manera, el sistema aprovecha mejor los recursos del procesador, ya que múltiples rutas se analizan simultáneamente sin depender unas de otras.

La recursión se detiene cuando se alcanzan carpetas que no contienen más subniveles. En ese punto, los resultados comienzan a consolidarse de manera ascendente, integrándose en una estructura compartida que almacena todos los archivos encontrados. Este proceso garantiza que, independientemente del nivel de profundidad del sistema de archivos, todos los resultados se unifiquen correctamente.

Aplicación en la búsqueda de archivos

En la búsqueda de archivos, esta estrategia se implementa directamente mediante un método recursivo (`ProcesarParalelo`). Cada vez que se detecta una subcarpeta, se genera una nueva tarea dentro de un `Parallel.ForEach`, permitiendo que múltiples subdirectorios se procesen al mismo tiempo.

Así, cada hilo del sistema se encarga de recorrer una rama distinta del árbol de directorios, mientras que los archivos encontrados se almacenan en una estructura concurrente (`ConcurrentBag`), evitando conflictos entre hilos.

Aplicación en la detección de duplicados

En la detección de duplicados, aunque no se utiliza recursión explícita, se mantiene la idea de descomposición del problema. Primero, el conjunto de archivos se divide en grupos según su tamaño, reduciendo el número de comparaciones necesarias. Luego, cada grupo se procesa de forma independiente, donde el cálculo de hashes se ejecuta en paralelo.

Este enfoque permite tratar cada grupo como un subproblema independiente, replicando el principio de la descomposición recursiva: dividir, procesar en paralelo y luego unificar resultados. Finalmente, los archivos que comparten el mismo hash se agrupan como duplicados.

Herramientas y tecnologías empleadas (C#, TPL, etc.).

- C#
- .NET
- Aplicación de consola (CLI)
- Task Parallel Library (TPL)
- `Parallel.ForEach`
- `ParallelOptions` (`MaxDegreeOfParallelism`)
- `ConcurrentBag`
- `ConcurrentDictionary`
- `lock` (sincronización)
- LINQ (`GroupBy`, `Where`, `OrderBy`)
- `System.IO` (manejo de archivos y directorios)
- `FileInfo` / `DirectoryInfo`
- SHA256 (hashing de archivos)
- `Stopwatch` (medición de tiempo)
- Programación asíncrona (`Task`, `async/await`)

5. Implementación Técnica

Descripción de la estructura del proyecto

El proyecto está organizado en una solución Visual Studio con dos carpetas principales: `src`, que contiene el proyecto ejecutable `ParallelStorageAnalyzer`, y `tests`, destinada a las pruebas automatizadas.

Dentro de `src`, el código se distribuye en cinco clases con responsabilidades bien delimitadas: `Program`, `ConsoleUI`, `BuscadorArchivo`, `DetectorDuplicados`, `ResultadoBusqueda` y `Metrics`. Este diseño sigue un esquema en capas donde cada clase tiene una única responsabilidad, facilitando las pruebas y el mantenimiento.

Explicación del código clave

Descomposición recursiva paralela (`BuscadorArchivo`)

El núcleo del sistema está en `ProcesarCarpeta`, que combina recursión con `Parallel.ForEach` para explorar el árbol de directorios simultáneamente:

Detección de duplicados (`DetectorDuplicados`)

Aplica una estrategia en dos fases: primero agrupa por tamaño para descartar candidatos obvios, luego calcula SHA-256 únicamente sobre los grupos restantes, reduciendo el trabajo costoso de hashing al mínimo necesario.

Uso de mecanismos de sincronización

El sistema evita condiciones de carrera mediante dos estructuras de `System.Collections.Concurrent`:

`ConcurrentBag<FileInfo>` se usa en `BuscadorArchivo` porque la escritura concurrente no requiere orden. Internamente mantiene colas locales por hilo.

`ConcurrentDictionary<string, List<FileInfo>>` se usa en `DetectorDuplicados` para asociar hashes con rutas. Las operaciones como `GetOrAdd` garantizan que dos hilos que calculen el mismo hash simultáneamente no pierdan ninguna entrada.

No se utilizan lock de forma explícita porque las colecciones concurrentes de .NET internalizan ese manejo, reduciendo la complejidad del código y el riesgo de deadlocks.

Justificación técnica de las decisiones tomadas

Descomposición recursiva. Recorrer el árbol de directorios de forma iterativa procesa los nodos de a uno. La recursión paralela lanza nuevas tareas TPL en cada nivel, permitiendo que ramas independientes del sistema de archivos se exploren simultáneamente.

SHA-256 sobre MD5. Aunque MD5 es más rápido, SHA-256 produce hashes de 256 bits que prácticamente eliminan colisiones falsas, garantizando que dos archivos marcados como duplicados sean realmente idénticos en contenido.

Environment.ProcessorCount como límite de paralelismo. Fijar un número estático de hilos (por ejemplo, 8) haría que el sistema fuera ineficiente en máquinas con menos núcleos o desperdiciara el hardware en máquinas con más. Usar el valor dinámico del sistema garantiza el mejor aprovechamiento posible sin configuración manual.

Manejo de **UnauthorizedAccessException**. El sistema de archivos de Windows contiene directorios del sistema protegidos. Capturar y descartar esta excepción permite que el escaneo continúe en lugar de terminar abruptamente, ofreciendo resultados válidos en lugar de un error fatal.

6. Evaluación de Desempeño

Comparativa entre ejecución secuencial y paralela

Para un análisis efectivo, reunimos los resultados obtenidos en las pruebas realizadas sobre el mismo conjunto de datos. Un speedup por encima de la cantidad de núcleos se conoce como speedup superlineal. Esto es posible porque al usar más núcleos tenemos más memoria cache disponible y el acceso a cache es mucho más rápido que el acceso a memoria RAM.

Núcleos	Tiempo Paralelo (ms)	Tiempo Secuencial (ms)	Speedup	Eficiencia
2	10,989	23,076	2.10x	105.0%
4	8,484	23,564	2.78x	69.4%
8	7,275	23,267	3.20x	40.0%
16	7,684	23,788	3.10x	19.4%

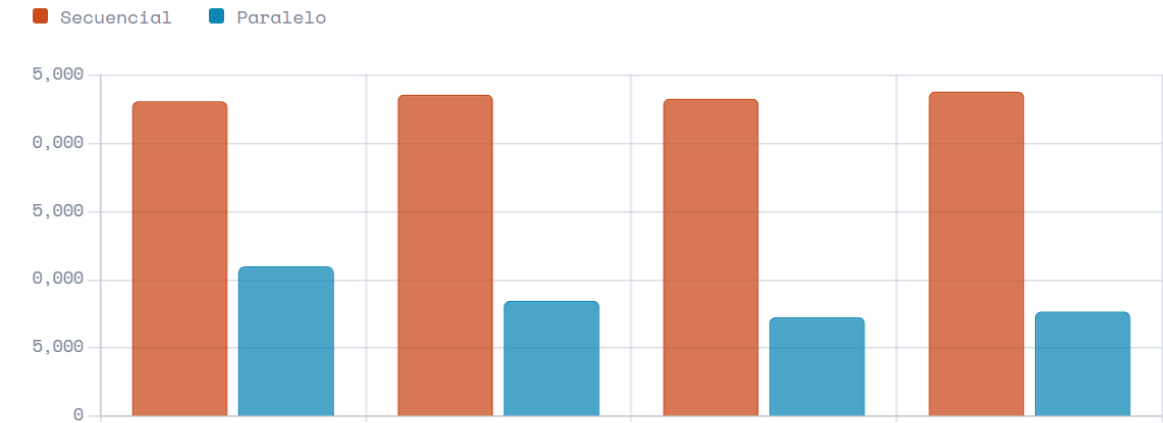
Métricas: tiempo de ejecución, eficiencia, escalabilidad

En la tabla vemos que al duplicar los núcleos de 2 a 4, el tiempo mejora, pero no se reduce a la mitad. Al llegar a los 16 núcleos, el tiempo incluso tiende a estancarse.

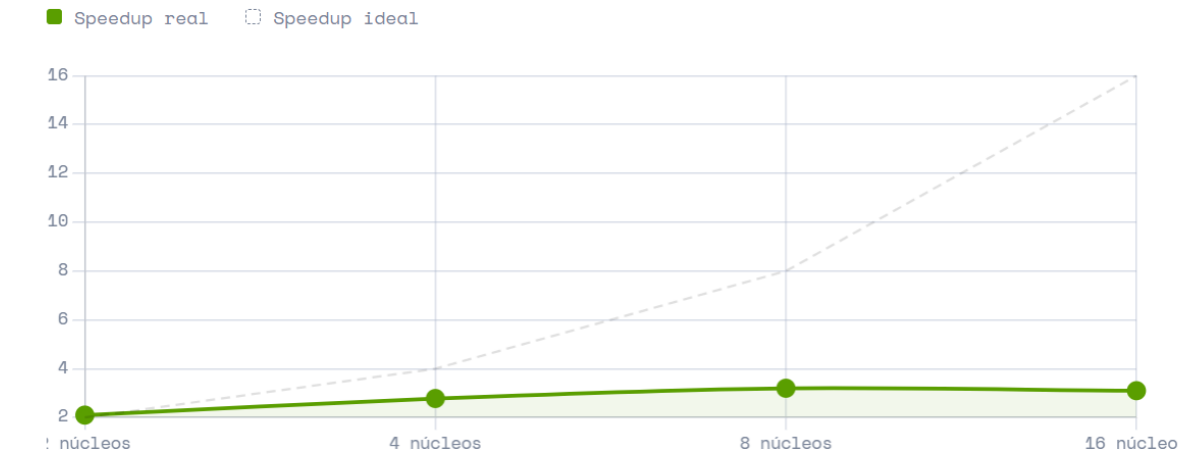
El sistema alcanza su rendimiento óptimo cerca de los 8 núcleos. A partir de ahí, el costo de gestionar los hilos es mayor que la velocidad que aportan.

Gráficas

TIEMPO DE EJECUCIÓN
Paralelo vs Secuencial (ms)



ACELERACIÓN
Speedup por núcleos



Análisis de cuellos de botella o limitaciones

Los resultados confirman que no importa cuántos núcleos añadamos, la velocidad de respuesta está dictada por la tasa de transferencia de datos del disco.

Incluso con 16 núcleos, el Speedup se estancó en $\sim 3x$. Esto sugiere que aproximadamente el 33% del código es intrínsecamente secuencial, limitando el beneficio total del paralelismo.

7. Trabajo en Equipo

Descripción del reparto de tareas

Para garantizar el éxito del proyecto Parallel Storage Analyzer, se distribuyeron las responsabilidades basándose en los pilares fundamentales del desarrollo: procesamiento de datos, gestión de interfaz y control de calidad.

La distribución de tareas se realizó de la siguiente manera:

Rubby Keyther: Desarrollo de la clase DetectorDuplicados, implementando comparación usando hashing. Responsable de la lógica de búsqueda de duplicados de forma secuencial y paralela, y de la implementación del método secuencial en BuscadorArchivo.

Eidan Then: Diseño e implementación de la lógica del Dashboard para la eliminación de archivos y grupos duplicados. Creación de la clase FlujoPrincipalTests para pruebas unitarias y automatizadas.

Jeremy Reyes: Implementación de la lógica de búsqueda paralela en BuscadorArchivo. Diseño y desarrollo de la interfaz de usuario principal, y creación de la clase Metrics para el cálculo del speedup y la eficiencia.

Herramientas utilizadas para coordinación (Git, AzureDevops, Trello, Excel, etc.)

La coordinación del equipo se llevó a cabo usando las siguientes herramientas:

Grupo de WhatsApp: Utilizado como canal principal para la resolución de dudas rápidas, asignación de tareas y notificación de errores y cambios en el repositorio.

Microsoft Teams: Se realizaron reuniones de revisión para documentar los avances y realizar revisiones del código. Estas reuniones fueron clave para explicar los cambios en la lógica de paralelismo y asegurar la integridad del código.

8. Conclusiones

Principales aprendizajes técnicos

El desarrollo de este sistema permitió comprender de forma práctica el uso del paralelismo en aplicaciones reales.

Se afianzó el uso de la **Task Parallel Library (TPL)**, especialmente con `Parallel.ForEach` y el control del paralelismo mediante `MaxDegreeOfParallelism`. Asimismo, se adquirió experiencia en el manejo de estructuras concurrentes como `ConcurrentBag` y `ConcurrentDictionary`, fundamentales para evitar conflictos entre hilos.

Otro aprendizaje clave fue la implementación de la **descomposición recursiva**, permitiendo dividir un problema complejo en subproblemas más manejables y paralelizables. Además, se incorporó la medición de rendimiento mediante `Stopwatch`, lo que permitió analizar métricas como **speedup** y **eficiencia**.

Retos enfrentados y superados

Se presentaron dificultades al integrar todas las partes del sistema (búsqueda, detección de duplicados, interfaz de usuario y métricas), las cuales fueron superadas mediante una correcta organización modular del código.

Otro desafío importante fue comprender el funcionamiento lógico de la detección de duplicados mediante hashes, ya que implica un proceso en dos etapas: primero agrupar archivos por tamaño para reducir el espacio de búsqueda, y luego calcular y comparar sus hashes (SHA256) para determinar si realmente son idénticos. Entender esta lógica fue fundamental para implementar una solución eficiente y correcta.

Comprender a fondo el uso de `Parallel.ForEach` y el manejo de la colección `ConcurrentBag<FileInfo>` también resultó un desafío. Sin embargo, a medida que se profundizó en su funcionamiento, se logró aplicar correctamente el paralelismo en la exploración de carpetas.

Posibles mejoras o líneas futuras

Se podría mejorar la experiencia del usuario implementando una **interfaz gráfica (GUI)**, facilitando la visualización de resultados y la interacción con el sistema.

Otra línea de mejora sería incorporar la capacidad de **cancelar procesos en ejecución**, especialmente útil en análisis de grandes volúmenes de datos.

Asimismo, se podría extender el sistema para realizar **análisis más avanzados del almacenamiento**, como identificar archivos innecesarios, generar reportes o estadísticas de uso del disco.

9. Referencias

Fuentes bibliográficas, técnicas o académicas consultadas

<https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/lambda-expressions-in-plinq-and-tpi>

<https://learn.microsoft.com/en-us/dotnet/api/system.io.fileinfo.name?view=net-10.0>

<https://learn.microsoft.com/en-us/dotnet/api/system.collections.concurrent.concurrentbag-1?view=net-10.0>

<https://learn.microsoft.com/es-es/dotnet/visual-basic/programming-guide/concepts/linq/how-to-query-for-the-largest-file-or-files-in-a-directory-tree>

<https://learn.microsoft.com/es-es/dotnet/visual-basic/programming-guide/concepts/linq/how-to-query-for-duplicate-files-in-a-directory-tree-linq>

10. Anexos

Manual de ejecución del sistema

Requisitos previos

Antes de ejecutar la aplicación, asegúrese de tener instalado:

.NET 8.0 SDK o superior

Visual Studio 2022 (versión 17.x)

Sistema operativo: Windows, Linux o macOS

Obtener el proyecto

Clone el repositorio o descargue el código fuente

bash:

git clone <https://github.com/JeremyRG32/ParallelStorageAnalyzer.git>

Compilar el proyecto

Visual Studio 2022

1. Abra el archivo ParallelStorageAnalyzer.sln
2. En el menú superior seleccione **Build** → **Build Solution** (o Ctrl+Shift+B)
3. Verifique que no haya errores en la ventana de salida

Ejecutar la aplicación

Desde Visual Studio: presione F5 para ejecutar con depurador, o Ctrl+F5 sin él.

Uso paso a paso

Al iniciar, la aplicación guía al usuario mediante tres preguntas en consola:

Paso 1 — Ingresar la ruta a escanear

Ingrese la ruta a escanear (ej: C:\Windows): C:\Users\Usuario\Documents

El sistema valida que la ruta exista y no esté vacía. Si es inválida, lo notifica y vuelve a solicitar la entrada.

Paso 2 — Definir el tamaño mínimo

Ingrese el tamaño mínimo de archivos a reportar en MB: 50

Solo se reportarán archivos que superen ese umbral. Debe ser un número entero mayor o igual a cero.

Paso 3 — Seleccionar el modo de búsqueda

Seleccione el modo de busqueda

1. Paralelo
2. Secuencial
3. Comparativa de Ambos modos

El modo paralelo aprovecha todos los núcleos del procesador. El modo secuencial procesa carpeta por carpeta de forma tradicional y el modo comparativa realiza la búsqueda en ambos modos y devuelve las métricas.

Interpretar los resultados

Una vez completado el escaneo, la consola muestra un listado ordenado de mayor a menor tamaño

Si no se encuentran archivos que superen el tamaño indicado, el sistema muestra:

No se encontraron archivos con el tamaño especificado

Ejecutar las pruebas

bash

dotnet test

Los resultados indicarán cuántas pruebas pasaron, fallaron o fueron omitidas.

Capturas adicionales, pruebas complementarias

Dashboard de Archivos encontrados

#	Tamaño	Nombre	Ruta
1	637.79 MB	faf506e.msp	C:\Windows\Installer
2	601.00 MB	setupact.log	C:\Windows\Panther
3	598.83 MB	6e346b24.msp	C:\Windows\Installer
4	467.18 MB	3c14c49b.msp	C:\Windows\Installer
5	418.03 MB	116c595a.msp	C:\Windows\Installer
6	369.08 MB	52a95492.msp	C:\Windows\Installer
7	354.25 MB	3dc7e.msi	C:\Windows\Installer
8	336.57 MB	9306b328.msp	C:\Windows\Installer
9	321.88 MB	31a3f036.msp	C:\Windows\Installer
10	294.84 MB	msedge.dll	...6ad364e35_10.0.26100.8246_none_2ec91fd13ab0f039
11	294.84 MB	msedge.dll	C:\Windows\System32\Microsoft-Edge-WebView
12	293.09 MB	msedge.dll	...6ad364e35_10.0.26100.8036_none_2eddbf5f3aa19f00
13	286.91 MB	e378c2a.msp	C:\Windows\Installer
14	282.88 MB	211c70a6.msp	C:\Windows\Installer
15	276.91 MB	7617f235.msp	C:\Windows\Installer
16	251.35 MB	msedge.dll	...3856ad364e35_10.0.26100.1_none_8fe1c356f34a19ca
17	244.05 MB	2ddc5a9e.msp	C:\Windows\Installer
18	235.68 MB	29082.msi	C:\Windows\Installer
19	226.18 MB	MigLog.xml	C:\Windows\Panther
20	223.79 MB	289ba14a.msp	C:\Windows\Installer
21	208.14 MB	MRT.exe	C:\Windows\System32

Total: 1 grupo(s) de duplicados encontrados.
? 2 copias de: msedge.dll

Tiempo de ejecución (Paralelo): 6404ms
Núcleos Utilizados: 16

¿Qué deseas hacer?

1. Buscar archivos de nuevo
2. Eliminar un archivo
3. Eliminar archivos duplicados
4. Salir

Opción:

Dashboard de Metricas

COMPARATIVA DE MÉTRICAS (I/O)		
Métrica	Secuencial	Paralelo
Tiempo Total (ms)	21491 ms	6010 ms
Archivos Procesados	21	21
Grupos Duplicados	1	1
Núcleos Utilizados	1	16
Speedup Logrado		3.58x
Eficiencia Real		22.3%

Presione cualquier tecla para continuar...

Enlace al repositorio de Git (público)

<https://github.com/JeremyRG32/ParallelStorageAnalyzer.git>